CHARUSAT®
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 9: Graph Traversal: Depth-First Search and Breadth-First Search

Practical Aim:

To implement depth-first and breadth-first traversal on a graph and develop the ability to reason about how the order of exploration changes depending on the traversal strategy used.

Problem 1 :

A disaster relief team needs to inspect all buildings in a city after an earthquake. The city is mapped as a network where buildings are connected by roads. One team starts at a building and goes as deep into each road as possible before backtracking, while a second team spreads out level by level — first checking all buildings directly connected to the start, then all buildings two roads away, and so on. Given the city map as a graph and a starting building, implement both exploration strategies and print the order in which buildings are visited by each team.

Describe how the order of buildings visited differs between the two strategies on the same graph. What data structure does each strategy rely on to decide which building to visit next, and how does that structure cause the difference in order?

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practical-9

Output:

```
PS C:\Users\ADMIN\OneDrive\Desktop\DSA> cd "c:\Users\ADMIN\OneDrive\Desktop\DSA\Practical-9\" ; if ($?) { g++ PRA
9-1.cpp -o PRA9-1 } ; if ($?) { .\PRA9-1 }
Enter number of buildings: 7
Enter number of roads: 6
Enter roads (u v):
0 1
0 2
1 3
1 4
2 5
2 6
Enter starting building: 0
DFS Order: 0 1 3 4 2 5 6
BFS Order: 0 1 2 3 4 5 6
PS C:\Users\ADMIN\OneDrive\Desktop\DSA\Practical-9>
```

Problem Faced:

The main challenge was representing the city map as a graph (using an adjacency list) and implementing both traversals correctly: DFS using recursion (or an explicit stack) to go as deep as possible before backtracking, and BFS using an explicit queue to expand level by level, while using a visited array in both so that no building is inspected twice and the traversal terminates correctly.

Outcome/Conclusion:

Depth-first search was implemented recursively, visiting an unvisited neighbour and immediately recursing into it before considering the next neighbour, which naturally goes as deep as possible along one road before backtracking. Breadth-first search was implemented using a queue, starting by enqueuing the starting building, then repeatedly dequeuing a building, printing it, and enqueuing all of

its unvisited neighbours, which visits all buildings one road away before moving to buildings two roads away. Both were run on the same city graph from the same starting building, and the visiting order produced by each was printed.

The visiting order differs because DFS commits to one path and keeps extending it until it cannot go further, then backtracks to try other branches, so it can visit a building far from the start very early if it lies along the first path explored. BFS instead visits buildings strictly in order of their distance (in number of roads) from the start, so it always finishes checking everything at one distance before moving further out. This difference comes directly from the data structure each strategy relies on: DFS relies on a stack (the recursive call stack, or an explicit stack if written iteratively), which always continues from the most recently discovered building, producing depth-first order; BFS relies on a queue, which always continues from the earliest discovered building still waiting, producing breadth-first, level-by-level order.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation:

[Problem 1] Both strategies visit every reachable node exactly once. If the goal is simply to check whether a specific building exists in the network, which strategy would find it sooner if the building is very close to the start? What if it is very far and deep? Explain your reasoning without tracing through any specific example.

Sol: If the target building is very close to the start (reachable in just one or two roads), BFS is likely to find it sooner, because BFS systematically checks all buildings at distance one, then distance two, and so on, so a nearby building is guaranteed to be checked within the first few steps regardless of how the graph is laid out elsewhere. DFS, on the other hand, might wander deep down an unrelated branch of the graph before ever coming back to check a nearby building, because it commits fully to one path before backtracking. If the target building is very far and deep along a particular path, DFS could find it sooner, since it plunges directly along a path without first exhausting all the shallower buildings the way BFS does; however, this is not guaranteed, because DFS's speed depends entirely on whether the branch it chooses to explore first happens

to lead toward the target — if it explores the wrong branch first, it may take just as long or longer than BFS.

[Problem 1] Your DFS implementation likely uses recursion. What real-world constraint on the computer could cause your recursive DFS to fail on a very large city map, and how would you restructure your solution to avoid that failure?

Sol: A recursive DFS makes one function call per node along the deepest path it explores, and each call uses a frame on the program's call stack. On a very large or very deeply connected city map, the recursion could go so deep that it exceeds the call stack's size limit set by the operating system or runtime, causing a stack overflow and crashing the program, even though there may be plenty of heap memory available. To avoid this, the DFS can be restructured iteratively using an explicit stack data structure stored on the heap instead of relying on the call stack: the starting building is pushed onto the stack, and then the program repeatedly pops a building, processes it if not already visited, and pushes its unvisited neighbours, which produces the same depth-first order without being limited by the call stack's fixed size.